Autenticación y decoradores

01 Sesiones y cuentas

Biblioteca usa autenticación de Django con vistas de función. El modelo de cuenta se obtiene mediante get_user_model(). Usuario guarda el perfil institucional y puede vincularse a la cuenta mediante OneToOneField. Los préstamos apuntan a la cuenta de Django.

Piezas de configuración

sistema/settings.py habilita sesiones y autenticación mediante middleware. SessionMiddleware se ejecuta antes de AuthenticationMiddleware y este expone request.user. LOGIN_URL = "login" identifica la ruta de acceso. LOGIN_REDIRECT_URL = "index" está configurado. La vista personalizada elige de forma explícita el destino de éxito.

Inicio de sesión

GET /biblioteca/login/ presenta el formulario. POST entrega username y password a authenticate(). Si el backend devuelve una cuenta, login() establece su sesión y la vista redirige. Si falla, el formulario se vuelve a mostrar con un mensaje genérico. Si ya hay una sesión, la vista redirige directamente.

next viene del POST o de la consulta GET. La vista lo valida con url_has_allowed_host_and_scheme(), el host de la solicitud y la exigencia de HTTPS cuando corresponda. El formulario conserva un destino válido en un input oculto. La alternativa predeterminada es index.

Cerrar sesión

POST /biblioteca/logout/ llama logout() y redirige al login. GET recibe 405. La portada muestra un formulario de salida con token CSRF. El login y los formularios de escritura también incluyen ese token.

Autenticación y decoradores

02 Qué controla el acceso

Matriz de acceso vigente

Público. Portada, listado y detalle de libros, listado y detalle de usuarios y formulario de inicio de sesión.

Con sesión. Alta, edición y borrado de libros y perfiles. Registro de préstamos. Cada una de estas siete vistas tiene @login_required.

Por método. Logout acepta POST. La protección CSRF se aplica a las solicitudes correspondientes según la configuración de Django.

Autenticación, autorización y CSRF

La autenticación identifica a la cuenta. La autorización decide si esa cuenta puede ejecutar una operación concreta. CSRF protege formularios frente a envíos originados en sitios externos. @login_required comprueba request.user.is_authenticated. Cualquier cuenta autenticada puede acceder a las siete vistas protegidas por esa comprobación.

Datos públicos

El listado y el detalle de usuarios exponen RUT y correo institucional. La portada pública muestra información de prestatarios recientes, como nombre, correo y RUT cuando hay perfil. Si esos datos son internos, proteger solo /usuarios/ deja la exposición de la portada. La política debe abarcar ambas superficies.

Cuentas creadas desde el CRUD

usuario_crear usa User.objects.create() sin establecer una contraseña. El perfil y el estado activo por sí solos no habilitan el acceso por contraseña. Para una cuenta de prueba, usar createsuperuser en un entorno local o create_user() al preparar una cuenta ordinaria. Para una cuenta existente, changepassword permite asignar una clave por un canal administrativo controlado.

Autenticación y decoradores

03 Cómo añadir @login_required

El decorador envuelve la vista y examina la sesión antes de ejecutar su cuerpo. Se coloca inmediatamente encima de def, en biblioteca/views.py. Este ejemplo protege el listado de perfiles que hoy es público, si la política del proyecto exige una sesión para consultarlo.

```
from django.contrib.auth.decorators import login_required

@login_required
def usuario_listar(request):
    usuarios = (
        Usuario.objects.select_related("user")
        .order_by("-id_usuario")
    )
    return render(request, "biblioteca/usuario_list.html", {
        "usuarios": usuarios,
    })
```

El import ya existe en el proyecto. Conservar la URL de usuario_listar y dejar pública iniciar_sesion para que una cuenta anónima pueda abrirla. El decorador puede añadirse también a usuario_detalle si ambas rutas deben tener la misma política.

Qué observar

GET /biblioteca/usuarios/ sin sesión devuelve 302 hacia /biblioteca/login/?next=/biblioteca/usuarios/. Al iniciar sesión, el navegador vuelve al listado mediante GET. Si se intercepta un POST, el login conserva el destino, pero el navegador no reenvía automáticamente los datos de ese POST.

Se puede reflejar el estado con {% if user.is_authenticated %} en la plantilla. La comprobación del servidor se mantiene aunque el enlace se oculte. Probar la URL directa para verificar el control de acceso.

Si la portada debe seguir pública, retirar de su HTML los datos institucionales de préstamos para visitantes o separar el panel privado de los contadores públicos.

Autenticación y decoradores

04 Cómo añadir un permiso

Ejemplo propuesto para reservar la eliminación de libros a operadores autorizados. Django genera permisos por modelo al migrar. El código de eliminación de Libro es biblioteca.delete_libro.

```
from django.contrib.auth.decorators import (
    login_required, permission_required,
)

@login_required
@permission_required("biblioteca.delete_libro", raise_exception=True)
def libro_eliminar(request, pk):
    # Conservar el cuerpo actual de la vista.
    ...
```

Aplicación. Importar permission_required y añadir esa línea entre @login_required y def libro_eliminar. El fragmento usa puntos suspensivos para representar el cuerpo existente. Conservar ese cuerpo completo al editar la vista.

Asignación. En el administrador de Django, crear un grupo para operadores y asignarle el permiso de eliminación de libros. Añadir al grupo solo las cuentas autorizadas. Verificar con una cuenta ordinaria, ya que una supercuenta activa dispone de todos los permisos.

Por qué importa el orden

El decorador exterior, login_required, procesa primero al visitante anónimo y lo lleva a iniciar sesión. Con sesión, permission_required evalúa el permiso y responde 403 si falta. Con el permiso, la vista aplica sus reglas de negocio y todavía puede bloquear el borrado por ejemplares asociados.

Para restringir otras escrituras, usar el permiso correspondiente, por ejemplo biblioteca.add_libro, biblioteca.change_libro, biblioteca.change_usuario o biblioteca.add_prestamo. Revisar cada vista pública y privada antes de extender la política.

Autenticación y decoradores

05 Decorador propio y pruebas

Para reutilizar la misma política de borrado en varias vistas, se pueden componer los decoradores de Django en biblioteca/decorators.py. Para una sola vista, la versión explícita de la página anterior es suficiente.

```
from django.contrib.auth.decorators import (
    login_required, permission_required,
)


def puede_eliminar_libros(view_func):
    permiso = permission_required(
        "biblioteca.delete_libro", raise_exception=True,
    )
    return login_required(permiso(view_func))
```

Importar con from .decorators import puede_eliminar_libros en biblioteca/views.py y aplicar @puede_eliminar_libros encima de def libro_eliminar. Esta versión compone los dos controles de la página anterior y los sustituye en esa vista.

Verificación del cambio propuesto

1. Anónimo. GET a la ruta protegida debe devolver 302 con next. 2. Cuenta con sesión y sin permiso. GET y POST deben devolver 403. 3. Cuenta autorizada. GET muestra la confirmación y POST borra un libro sin ejemplares. 4. Con ejemplares. Un POST autorizado debe preservar el libro y mostrar el motivo.

Verificar también que un POST sin token CSRF sea rechazado en una prueba con Client(enforce_csrf_checks=True). Probar las vistas directamente, además de los botones de navegación.

Qué se ejecutó y dónde revisar

Las 16 pruebas existentes pasaron. Los cambios de permisos descritos son propuestas. Fuentes del repositorio: biblioteca/views.py, biblioteca/urls.py, biblioteca/models.py, biblioteca/tests.py, biblioteca/templates/biblioteca/login.html, biblioteca/templates/biblioteca/index.html y sistema/settings.py.

Autenticación y decoradores

01 Formulario de acceso

Fragmento completo del formulario que se coloca en biblioteca/templates/biblioteca/login.html. La vista iniciar_sesion entrega error y next.

```
<h1>Iniciar sesión</h1>
{% if error %}
  <p>{{ error }}</p>
{% endif %}
<form method="post" action="{% url 'login' %}">
  {% csrf_token %}
  <input type="hidden" name="next" value="{{ next|default:'' }}">
  <label for="username">Usuario</label>
  <input id="username" name="username" autocomplete="username"
         required>
  <label for="password">Contraseña</label>
  <input id="password" name="password" type="password"
         autocomplete="current-password" required>
  <button type="submit">Entrar</button>
</form>
```

Si el usuario visita /biblioteca/libros/crear/ sin sesión, Django redirige al login con next en la consulta. La vista valida el destino antes de colocarlo en el campo oculto. Al enviar el formulario, los nombres username, password y next coinciden con las claves que lee iniciar_sesion. El campo type="password" oculta el texto mientras se escribe.

Autenticación y decoradores

02 Procesar credenciales y destino

Versión didáctica del flujo de iniciar_sesion. Valida next antes de redirigir y conserva el comportamiento de la vista del proyecto.

```
def iniciar_sesion(request):
    destino = request.POST.get("next") or request.GET.get("next", "")
    valido = url_has_allowed_host_and_scheme(
        destino, allowed_hosts={request.get_host()},
        require_https=request.is_secure(),
    )
    if request.user.is_authenticated:
        return redirect(destino if valido else "index")

    error = ""
    if request.method == "POST":
        usuario = authenticate(
            request,
            username=request.POST.get("username", ""),
            password=request.POST.get("password", ""),
        )
        if usuario is not None:
            login(request, usuario)
            return redirect(destino if valido else "index")
        error = "El usuario o la contraseña no son correctos."

    return render(request, "biblioteca/login.html", {
        "error": error,
        "next": destino if valido else "",
    })
```

authenticate() comprueba las credenciales mediante el backend. login() vincula la cuenta con la sesión. En la siguiente solicitud, AuthenticationMiddleware vuelve a identificarla mediante request.user. Los imports correspondientes ya se encuentran en biblioteca/views.py.

Autenticación y decoradores

03 Mostrar acceso y salida

Fragmento para la portada. El botón envía un POST a logout con CSRF. La rama para visitantes muestra un enlace GET al login.

```
{% if user.is_authenticated %}
  <p>Sesión de {{ user.username }}</p>
  <form method="post" action="{% url 'logout' %}">
    {% csrf_token %}
    <button type="submit">Cerrar sesión</button>
  </form>
{% else %}
  <a href="{% url 'login' %}">Iniciar sesión</a>
{% endif %}

# biblioteca/views.py
def cerrar_sesion(request):
    if request.method != "POST":
        return HttpResponseNotAllowed(["POST"])
    logout(request)
    return redirect("login")
```

El condicional controla qué se muestra en HTML. El decorador controla si el servidor ejecuta la vista. Puedes comprobarlo visitando directamente una URL protegida sin sesión, incluso cuando no aparece ningún enlace hacia ella.

Autenticación y decoradores

04 Añadir un decorador y verificarlo

Ejemplo completo de una vista existente a la que se puede aplicar @login_required. El cambio propuesto pide sesión para leer el detalle de un perfil.

```
from django.contrib.auth.decorators import login_required

@login_required
def usuario_detalle(request, pk):
    usuario = get_object_or_404(
        Usuario.objects.select_related("user"), pk=pk,
    )
    prestamos = (
        Prestamo.objects.filter(usuario=usuario.user)
        .select_related("id_ejemplar__id_libro")
        .order_by("-fecha_prestamo")
    )
    return render(request, "biblioteca/usuario_detail.html", {
        "usuario": usuario,
        "prestamos": prestamos,
    })
```

Con un perfil existente, abre /biblioteca/usuarios/7/ en una ventana privada y reemplaza 7 por su identificador. Debe redirigir al login con next. Inicia sesión y verifica que vuelva al detalle. Para restringir el acceso a operadores, combina el decorador con @permission_required("biblioteca.view_usuario", raise_exception=True), debajo de @login_required, y asigna el permiso a un grupo.